

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO E ANÁLISE DE GRAFOS

TAREFA 2 — TEORIA DOS GRAFOS (TEG)

1. Identificação da Equipe

- **Alunos:** Caio Veiga Maievski e Paulo Henrique Gomes de Senna
- **Disciplina:** Teoria dos Grafos (TEG)
- **Data:** 18/06/2026

2. Identificação da Tarefa

Tarefa 2: Implementação de um grafo com carga primária a partir de um arquivo de base de palavras. O objetivo principal é construir um sistema em linguagem C para representar um grafo não direcionado ponderado onde os vértices são palavras em português de tamanho fixo (4 letras) e as arestas conectam palavras que diferem por apenas uma letra (ex: MATA → MALA). O sistema deve analisar a topologia do grafo (graus, componentes conexos, classificação estrutural) e determinar caminhos mínimos utilizando o algoritmo de Dijkstra com fila de prioridade (Min-Heap).

3. Explicação Conceitual sobre a Solução

A arquitetura do software foi desenvolvida visando alta performance e total fidelidade aos requisitos matemáticos estabelecidos:

- **Representação do Grafo:** Implementou-se uma estrutura de Lista de Adjacências utilizando alocação dinâmica de memória. Cada palavra da base corresponde a um vértice indexado, contendo um ponteiro para uma lista encadeada de nós adjacentes (Aresta), minimizando o desperdício de memória associado a matrizes em grafos esparsos.
- **Tratamento e Carga de Dados:** O arquivo de entrada `base_de_palavras_4_letras.txt` é processado linha por linha via `fgets`. A rotina de higienização limpa caracteres ocultos (`\r\n`) e converte todas as cadeias para maiúsculas (`toupper`), neutralizando inconsistências de caixa.

- **Mapeamento de Adjacências:** Executado em complexidade de tempo quadrática $O(n^2)$. Pares de vértices são comparados caractere por caractere; caso a divergência posicional seja exatamente igual a 1, uma aresta bidirecional com peso unitário é estabelecida. Durante este processo, o sistema monitora a existência de laços ou arestas múltiplas para classificar a estrutura.
- **Busca de Componentes Conexos:** Desenvolveu-se uma Busca em Largura (BFS) iterativa que explora o grafo de forma exaustiva, agrupando vértices mutuamente alcançáveis e calculando os graus máximos e mínimos internos de cada subgrafo isolado.
- **Algoritmo de Dijkstra Otimizado:** Para computar a menor sequência de transformações entre duas palavras fornecidas pelo usuário, implementou-se o algoritmo de Dijkstra estruturado sobre uma Fila de Prioridade (*Min-Heap*) binária alocada dinamicamente, garantindo operações de extração de custo mínimo em tempo logarítmico.

4. Instruções sobre a Compilação e Execução

O código foi projetado em conformidade com o padrão ANSI C puro, dispensando quaisquer dependências de bibliotecas externas adicionais:

1. **Compilador Recomendado:** GCC (GNU Compiler Collection).
2. **Comando para Execução:**
`./grafo_palavras base_de_palavras_4_letras.txt`

5. Descrição e Análise de Resultados

O processamento completo da base de dados gerou um modelo matemático com propriedades topológicas de grande interesse analítico:

1. **Dimensão do Grafo:** Foram identificados e indexados 1604 vértices (palavras) e estabelecidas 7390 arestas válidas de transformação de 1 caractere.
2. **Classificação Estrutural:** O sistema acusou a presença de 0 laços e 0 arestas duplas. Desse modo, a estrutura classifica-se rigorosamente como um **Grafo Simples**.
3. **Mapeamento de Conectividade:** O algoritmo de varredura identificou que o grafo é desconexo, contendo exatamente **28 componentes conexos** distintos.
4. **Fenômeno do Componente Gigante:** Os dados empíricos demonstram claramente a existência de um "Componente Gigante" (Componente 1) contendo 1575 das 1604 palavras (98,2% do total da base). Isso comprova que quase todo o vocabulário de 4 letras está densamente interconectado, permitindo caminhos de transformação contínuos entre termos aparentemente distantes. As demais 29 palavras distribuem-se em pequenos subgrafos isolados ou nós órfãos.

Componente	Qtd. Vértices	Vértice Central (Grau Máx)	Vértice Mínimo (Grau Mín)	Natureza e Membros do Subgrafo
Componente 1	1575	CAIA (Grau 26)	ABEL (Grau 1)	Componente Gigante contendo a quase totalidade do léxico interconectado por transições de caminhos mínimos.
Componente 9	2	IMPA (Grau 1)	IMPA (Grau 1)	Par de palavras isolado. Composto exclusivamente por: IMPA e IMPO.
Componente 17	2	BROA (Grau 1)	BROA (Grau 1)	Par de palavras isolado. Composto exclusivamente por: BROA e PROA.
Componentes 2-8, 10-16, 18-28	1 (cada)	Grau 0	Grau 0	Total de 25 palavras órfãs na base (não possuem nenhuma transição válida de 1 letra). Exemplos: BAAL, CNPQ, DION, EDEN, EGEU, ESAU, ETNA, JEAN, OGUM, UFRJ, UTAH, YORK, AZUL, BITS, EMIR, EXPO, FLOR, HOJE, ICOU, NOEL, PNEU, PSIU, SIRI, UTIL, ZEBU.

Validação Matemática de Consistência: A soma do tamanho de todas as partições isoladas mapeadas pelo algoritmo fecha perfeitamente a cardinalidade do conjunto de vértices total:

$$1575 + 2 + 2 + (25 \times 1) = 1604 \text{ Vértices Mapeados}$$

A perfeita equivalência entre a soma das partições e a carga primária inicial válida a consistência da busca em largura e assegura a integridade científica e analítica do sistema implementado.

6. Conclusão

A implementação utilizando listas de adjacência permitiu representar o grafo de forma eficiente, enquanto os algoritmos de BFS e Dijkstra possibilitaram a análise de sua conectividade e a determinação dos caminhos mínimos entre palavras.

Os resultados obtidos mostraram que a maior parte dos vértices pertence a um único componente conexo de grande dimensão, evidenciando um alto grau de interligação entre as palavras da base.